

CodeTober 2026

Data Engineering avec Python

20 défis • 5 jours par semaine • 45 min à 2 h • un projet fil rouge

Objectif

Passer progressivement d'une pratique d'intégration très orientée ETL/Talend à des réflexes de Data Engineering modernes : pipelines Python, APIs, SQL, qualité de données, chargements incrémentaux, observabilité, optimisation et modélisation analytique.

Le principe n'est pas de réapprendre le métier d'intégration, mais de transposer les compétences existantes vers des outils et architectures plus modernes.

Octobre 2026

Le principe du défi

Chaque jour ouvré apporte un problème concret, suffisamment challengeant pour demander de réfléchir, mais calibré pour rester réalisable en autonomie. Les énoncés détaillés ne figurent volontairement pas dans cette roadmap : ils seront découverts au fil du mois.

Autonomie	Chercher, expérimenter et coder avant de regarder une solution.
Concret	Manipuler des fichiers, APIs, bases et pipelines plutôt que résoudre des exercices abstraits.
Progressif	Réutiliser les acquis des jours précédents et augmenter doucement la complexité.
Professionnel	Travailler aussi la robustesse, les tests, les logs, la maintenabilité et les performances.
Accessible	Privilégier Python, sa bibliothèque standard et des outils gratuits/légers installables avec pip.

Rythme

Octobre 2026 commence un jeudi. Les 1er et 2 octobre servent donc de mise en route, puis le rythme devient lundi-vendredi. Le mois compte 22 jours ouvrés : 20 défis sont prévus, avec 2 journées joker pour rattraper, refactorer, approfondir ou simplement absorber une semaine chargée.

45 minutes = objectif minimal réussi. 2 heures = plafond. Si un défi exige régulièrement davantage, il doit être recalibré.

La progression du mois

Semaine 1 - Fondations d'un pipeline robuste

Repartir de problèmes familiers d'intégration, mais les traiter en Python proprement.

Nettoyage et validation d'un CSV imparfait • Traitement de gros fichiers par chunks / streaming • Architecture Extract - Transform - Load • Logging, métriques et gestion d'erreurs • Robustesse, reprise et idempotence

Semaine 2 - Ingestion moderne & chargements incrémentaux

Quitter le simple traitement de fichiers pour construire une vraie ingestion de données.

Consommation d'une API REST et pagination • Rate limiting, retry et backoff • Stockage SQL léger avec SQLite • Chargement incrémental / watermark • Déduplication et stratégie de clés

Semaine 3 - Qualité, performance & moteurs analytiques

Commencer à raisonner comme sur un pipeline destiné à la production.

Tests et contrôles de qualité de données • Détection et gestion du schema drift • Parallélisation avec concurrent.futures • Profiling mémoire / temps et optimisation • Découverte de DuckDB et du SQL analytique

Semaine 4 - Industrialisation & architecture

Assembler les briques et rendre le système rejouable, configurable et maintenable.

Mini data warehouse : faits et dimensions • Backfill et exécution paramétrée par date • Configuration des pipelines via YAML/JSON • Diagnostic d'un pipeline volontairement cassé • Pipeline final de bout en bout

Le fil rouge

Plusieurs défis alimenteront progressivement le même petit système data. L'idée est d'éviter une collection de scripts jetables et de terminer le mois avec une architecture cohérente et montrable.

API / fichiers → Raw data → Validation → Transformations → SQL / DuckDB

Les 20 défis - vue d'ensemble

#	Défi	Compétence centrale	Durée
1	The Dirty CSV	Nettoyage & qualité	~1 h
2	Chunk Me If You Can	Streaming / generators	1-1 h 30
3	Build Your Pipeline	Architecture ETL	~1 h 30
4	Houston, We Have Logs	Observabilité	~1 h
5	Everything Is Broken	Robustesse / idempotence	~1 h 30
6	API Hunter	REST / JSON	~1 h
7	Rate Limit Hell	Retry / backoff	~1 h 30
8	Welcome to SQLite	SQL / stockage	~1 h
9	Incremental Load	Watermark / delta	~1 h 30
10	Duplicate Killer	Déduplication	~1 h
11	Trust No Data	Tests / validation	~1 h 30
12	Schema Drift	Évolution de schéma	~1 h 30
13	Parallel Universe	Parallélisation	1-2 h
14	Memory Matters	Profiling / optimisation	~1 h 30
15	Enter DuckDB	SQL analytique	1-2 h
16	Mini Data Warehouse	Modélisation	~2 h
17	Back to the Future	Backfill / partitionnement	~1 h 30
18	Config, Not Code	Configuration	~1 h 30
19	Production Broke at 3 AM	Debugging	~1 h 30
20	The Final Pipeline	Architecture de bout en bout	~2 h

Stack envisagée

Socle : Python, bibliothèque standard, SQL. Selon les défis : pandas, requests/httpx, pytest, SQLite, DuckDB et éventuellement YAML. Les outils lourds ou nécessitant des comptes cloud ne sont pas indispensables au parcours.

Airflow, dbt, Spark, Kafka et les architectures cloud pourront être abordés conceptuellement lorsque cela apporte quelque chose, sans transformer CodeTober en concours d'installation de dépendances.

À la fin du mois

Le résultat attendu n'est pas seulement d'avoir écrit davantage de Python : c'est d'avoir acquis une vision plus moderne du Data Engineering et un petit projet capable de démontrer ingestion, qualité, transformations, stockage, observabilité, rejouabilité et bonnes pratiques d'industrialisation.